REVISION CHEAT SHEET:

TOPIC 19 OF 40

Resource Management and IDisposable in C#

Interview-ready notes · examples · C# · diagrams

Topic 19 of 40

C# has automatic memory management through the .NET garbage collector.

That does **not** mean every resource can simply be abandoned and left for the garbage collector.

An application may hold resources such as:

EXAMPLE / FLOW

```
File handles

Database connections

Network sockets

Streams

Operating-system handles

Locks

Unmanaged memory

Cryptographic handles

Native resources
```

These resources often need to be released at a predictable time.

That is the purpose of:

C#

```
IDisposable
```

and, for asynchronous cleanup:

C#

```
IAsyncDisposable
```

The central idea is:

> **Garbage collection manages managed memory. Disposal manages resource lifetime.**

Those are related concepts, but they are not the same thing.

Microsoft's .NET documentation explicitly notes that the garbage collector can reclaim managed memory but doesn't know how to release arbitrary unmanaged operating-system resources. `Dispose` provides deterministic cleanup for those resources.

High-Level Mental Model

EXAMPLE / FLOW

```
Create resource
    ↓
Use resource
    ↓
Finished with resource
    ↓
Dispose immediately
    ↓
Resource released
```

Compare that with relying on garbage collection:

EXAMPLE / FLOW

```
Create resource
    ↓
Stop referencing object
    ↓
Object becomes unreachable
    ↓
At some unknown future time
GC may collect it
```

For scarce resources such as:

EXAMPLE / FLOW

```
File handles

Sockets

Database resources
```

waiting for an unknown future GC cycle may be unacceptable.

Topic Coverage

This topic covers:

- managed vs unmanaged resources;
- garbage collection vs disposal;
- deterministic cleanup;
- `IDisposable`;
- `Dispose()`;
- `using`;
- using declarations;
- multiple disposable resources;
- disposal order;
- `try/finally`;
- object ownership;
- cascading disposal;
- idempotent `Dispose`;
- `ObjectDisposedException`;
- the dispose pattern;
- sealed classes;

- inheritable classes;
- `Dispose(bool disposing)`;
- finalizers;
- `GC.SuppressFinalize`;
- finalization cost;
- `SafeHandle`;
- `IAsyncDisposable`;
- `DisposeAsync`;
- `await using`;
- synchronous vs asynchronous disposal;
- types implementing both interfaces;
- ASP.NET Core DI disposal;
- scoped/transient/singleton ownership;
- resource leaks;
- common mistakes;
- performance;
- security;
- Fundraising examples;
- interview questions.

What Is a Resource?

A resource is something your application acquires and must eventually release.

Some resources are pure managed memory:

```
C#
Gift gift = new Gift();
```

Normally:

```
EXAMPLE / FLOW
No Dispose required
```

The garbage collector handles memory occupied by the object.

Other objects represent access to scarce external/native resources:

```
EXAMPLE / FLOW
FileStream

StreamReader

Database connection

Operating-system handle

Network stream
```

These commonly require deterministic cleanup.

Managed Memory

Consider:

```C#
Gift gift = new Gift();
```

Eventually:

```EXAMPLE / FLOW
gift is no longer reachable
    ↓
Garbage collector identifies object
    ↓
Managed memory reclaimed
```

You do not normally write:

```C#
Free(gift);
```

as you might in manually managed languages.

Unmanaged Resources

Examples include:

```EXAMPLE / FLOW
OS handles

Native memory

File descriptors

Window handles

Cryptographic native handles
```

The GC may know:

```EXAMPLE / FLOW
Managed wrapper object is unreachable
```

but not inherently:

```EXAMPLE / FLOW
How should this OS handle be closed?
```

That's why deterministic resource cleanup exists.

Garbage Collection vs Disposal

This distinction is essential.

Garbage Collection

Main responsibility:

```EXAMPLE / FLOW
Managed memory
```

Timing:

EXAMPLE / FLOW

```
Non-deterministic
```


You generally don't know exactly when GC will occur.

Disposal

Main responsibility:

EXAMPLE / FLOW

```
Resource cleanup
```

Timing:

EXAMPLE / FLOW

```
Deterministic
```

Your code explicitly says:

EXAMPLE / FLOW

```
I am finished with this resource now.
```

Mental Model

EXAMPLE / FLOW

```
GC
    →
"When is managed object memory reusable?"
```

EXAMPLE / FLOW

```
Dispose
    →
"When should this resource be released?"
```

IDisposable

The interface is conceptually simple:

C#

```
public interface IDisposable
{
    void Dispose();
}
```

A type implements it when it needs a cleanup contract.

Example:

C#

```
public sealed class GiftExportWriter :
    IDisposable
{
    public void Dispose()
    {
        // Release owned resources.
```

```
    }
}
```


`IDisposable.Dispose` represents application-defined cleanup such as releasing or resetting owned resources.

What Dispose Does Not Mean

`Dispose()` does not mean:

EXAMPLE / FLOW

```
Destroy this C# object immediately.
```

After:

C#

```
resource.Dispose();
```

the managed object may still exist in memory.

A reference may still point to it.

What has happened is:

EXAMPLE / FLOW

```
Owned resource lifetime
has ended
```

Example

C#

```
FileStream stream = File.OpenRead("gift-export.csv");

stream.Dispose();
```

The `stream` variable still exists.

But the underlying resource has been released.

Trying to perform an operation afterward may result in:

EXAMPLE / FLOW

```
ObjectDisposedException
```

depending on the type.

Deterministic Cleanup

Consider:

C#

```
FileStream stream = File.OpenRead("gift-export.csv");
```

You want:

EXAMPLE / FLOW

```
Open file
    ↓
Read file
```

```
    ↓
Close file
```

not:

EXAMPLE / FLOW

```
Open file
    ↓
Read file
    ↓
Hope GC eventually closes it
```

That predictable cleanup is called:

EXAMPLE / FLOW

```
Deterministic disposal
```

Manual Dispose

You could write:

C#

```
FileStream stream = File.OpenRead("gift-export.csv");

try
{
    // Use stream.
}
finally
{
    stream.Dispose();
}
```

This is safe because cleanup runs as control leaves the `try`.

But C# gives us a cleaner syntax:

EXAMPLE / FLOW

```
using
```

The `using` Statement

Example:

C#

```
using (
    FileStream stream =
        File.OpenRead(
            "gift-export.csv"))
{
    // Use stream.
}
```

When control leaves the block:

C#

```
stream.Dispose();
```

is automatically invoked.

The C# compiler implements `using` with `try/finally`-style semantics so disposal still occurs if the block exits because of an exception or a `return`.

Conceptual Translation

This:

```C#
using (
    FileStream stream = File.OpenRead(path)) { Process(stream);
}
```

is conceptually similar to:

```C#
FileStream stream = File.OpenRead(path);

try
{
    Process(
        stream);
}
finally
{
    stream?.Dispose();
}
```

The exact compiler transformation has language-specific details, but this is the right mental model.

Why `using` Is Better

It communicates:

```EXAMPLE / FLOW
This resource belongs to this scope.
```

You immediately know:

```EXAMPLE / FLOW
When it is acquired

Where it is used

When it is released
```

That improves reliability and readability.

using Declaration

Modern C# also supports:

```C#
using FileStream stream = File.OpenRead(path);

Process(
    stream);
```

No braces are required.

The resource is disposed when execution leaves the current scope.

Microsoft documents that a variable declared with a using declaration is disposed at the end of the scope in which it was declared.

Example

C#

```
public void ExportGift(
    string path)
{
    using StreamWriter writer = new(path);

    writer.WriteLine(
        "GiftId,Amount");

    writer.WriteLine(
        "1001,500.00");

    // writer.Dispose()
    // occurs when method scope ends.
}
```

using Block vs using Declaration

Block:

C#

```
using (
    StreamWriter writer = new(path)) { WriteGift(writer);
}
```

Resource lifetime:

EXAMPLE / FLOW

```
Only inside block
```

Declaration:

C#

```
using StreamWriter writer = new(path);

WriteGift(
    writer);

// More method code.
```

Resource lifetime:

EXAMPLE / FLOW

```
Until end of current scope
```

Prefer the Narrowest Useful Lifetime

Suppose:

```C#
using StreamWriter writer = new(path);

// 500 more lines of code.
```

Even if the writer is needed only for five lines, it remains alive until scope end.

A smaller block may be better:

```C#
using (
    StreamWriter writer = new(path)) { WriteGift(writer);
}
```

Then it is released sooner.

using Variables Are Readonly

A variable declared by a `using` statement/declaration cannot be reassigned or passed as `ref` / `out`.

For example:

```C#
using StreamReader reader = File.OpenText(path);

// reader = anotherReader; // Invalid.
```

The compiler protects the resource variable's disposal identity.

Multiple Resources

You can acquire multiple resources:

```C#
using (
    StreamReader first =
        File.OpenText(
            firstPath),
    second =
        File.OpenText(
            secondPath))
{
    // Use both.
}
```

Resources declared together are disposed in reverse declaration order.

Reverse Disposal Order

Conceptually:

EXAMPLE / FLOW

```
Create A
    ↓
Create B
    ↓
Use
    ↓
Dispose B
```

```
    ↓
Dispose A
```

This resembles stack-style ownership.

It is useful because later resources may depend on earlier ones.

Nested using

You may also write:

C#
```
using FileStream file = File.OpenRead(path);

using StreamReader reader = new(file);

string text = reader.ReadToEnd();
```

Scope exit conceptually:

EXAMPLE / FLOW
```
Dispose reader
    ↓
Dispose file
```

Resource Ownership

One of the most important resource-management questions is:

> **Who owns this object?**

Usually:

EXAMPLE / FLOW
```
The code that creates/acquires
a disposable resource
owns responsibility for disposal.
```

But not always.

Frameworks and dependency-injection containers can own resources for you.

Ownership Example

C#
```
using StreamWriter writer = new(path);
```

Your method created it.

Your method owns it.

Your method disposes it.

Borrowed Resource Example

C#
```
public void WriteGift(
    StreamWriter writer,
```

```
    Gift gift)
{
    writer.WriteLine(
        gift.GiftId);
}
```

Should `WriteGift` call:

C#
```
writer.Dispose();
```

?

Usually:

EXAMPLE / FLOW
```
No.
```

The method did not create the writer.

It merely borrowed it.

Ownership Rule

Useful default:

EXAMPLE / FLOW
```
Create it
    →
Own it

Own it
    →
Dispose it
```

But APIs can explicitly transfer ownership.

Document such behavior clearly.

Why Ownership Matters

Imagine:

C#
```
public void WriteGift(
    StreamWriter writer)
{
    writer.WriteLine(
        "Gift");

    writer.Dispose();
}
```

Caller:

C#
```
using StreamWriter writer = new(path);

WriteGift(
    writer);
```

```
writer.WriteLine(
    "Campaign");
```


Now the caller gets:

```
EXAMPLE / FLOW
ObjectDisposedException
```

because another method unexpectedly disposed a resource it did not own.

leaveOpen Pattern

Some .NET APIs allow:

```
EXAMPLE / FLOW
leaveOpen: true
```

when wrapping another resource.

Conceptually:

```
C#
using var wrapper = new SomeWrapper(stream, leaveOpen: true);
```

Then disposing:

```
EXAMPLE / FLOW
wrapper
```

does not dispose:

```
EXAMPLE / FLOW
stream
```

This makes ownership explicit.

Cascading Disposal

Suppose class A owns class B:

```
EXAMPLE / FLOW
A
    →
B
```

and B implements:

```
C#
IDisposable
```

Then A's `Dispose` should normally dispose B.

Microsoft explicitly recommends cascading disposal through owned resource containment.

Example


```C#
public sealed class GiftExportWriter :
    IDisposable
{
    private readonly StreamWriter
        _writer;

    public GiftExportWriter(
        string path)
    {
        _writer = new StreamWriter(path);
    }

    public void Dispose()
    {
        _writer.Dispose();
    }
}
```

Flow:

```EXAMPLE / FLOW
GiftExportWriter owns StreamWriter
    ↓
GiftExportWriter.Dispose()
    ↓
StreamWriter.Dispose()
```

Implement IDisposable Only When Needed

A normal service:

```C#
public sealed class GiftCalculator
{
    public decimal Calculate(
        decimal amount)
    {
        return amount;
    }
}
```

does not need:

```C#
IDisposable
```

There is no resource to release.

Do not implement disposal merely because:

```EXAMPLE / FLOW
All classes should clean themselves up.
```

Most managed classes require no custom disposal.

A Type Should Commonly Implement IDisposable When

It:

EXAMPLE / FLOW

```
Directly owns unmanaged resources
```

or:

EXAMPLE / FLOW

```
Owns managed objects
that themselves require disposal
```

or has another deterministic cleanup responsibility such as:

EXAMPLE / FLOW

```
Releasing a lock/registration

Removing an event registration

Returning a pooled resource
```

Microsoft notes that disposing can be used not only for native resources but also for cleanup such as releasing locks or removing owned registrations.

Simple Sealed IDisposable Class

For a sealed type that owns only managed disposable objects, a simple implementation is often enough:

C#

```
public sealed class GiftFileExporter :
    IDisposable
{
    private readonly StreamWriter
        _writer;

    private bool _disposed;

    public GiftFileExporter(
        string path)
    {
        _writer = new StreamWriter(path);
    }

    public void Write(
        Gift gift)
    {
        ObjectDisposedException
            .ThrowIf(
                _disposed,
                this);

        _writer.WriteLine(
            $"{gift.GiftId},{gift.Amount}");
    }

    public void Dispose()
    {
        if (_disposed)
        {
            return;
```

```
        }

        _writer.Dispose();

        _disposed = true;
    }
}
```

Dispose Should Be Idempotent

Idempotent means:

EXAMPLE / FLOW

```
Calling Dispose multiple times
should not repeatedly perform
destructive cleanup or fail unexpectedly.
```

Example:

C#

```
resource.Dispose();

resource.Dispose();
```

should normally be safe.

Microsoft's current dispose guidance recommends making disposal idempotent.

Typical Pattern

C#

```
public void Dispose()
{
    if (_disposed)
    {
        return;
    }

    // Cleanup.

    _disposed = true;
}
```

Using an Object After Disposal

After disposal, normal operational methods may throw:

C#

```
ObjectDisposedException
```

Example:

C#

```
private void ThrowIfDisposed()
{
    ObjectDisposedException.ThrowIf(
        _disposed,
```

```
        this);
}
```


Then:

```C#
public void WriteGift(
    Gift gift)
{
    ThrowIfDisposed();

    // Work.
}
```

Why ObjectDisposedException?

The object still exists.

But its usable resource lifetime has ended.

This is exactly what:

```EXAMPLE / FLOW
ObjectDisposedException
```

communicates.

The Full Dispose Pattern

The classic dispose pattern matters most when a class:

```EXAMPLE / FLOW
Can be inherited
```

or:

```EXAMPLE / FLOW
Directly manages unmanaged resources
```

Typical shape:

```C#
public class ResourceHolder :
    IDisposable
{
    private bool _disposed;

    public void Dispose()
    {
        Dispose(
            disposing: true);

        GC.SuppressFinalize(
            this);
    }

    protected virtual void Dispose(
        bool disposing)
    {
        if (_disposed)
        {
```

```
            return;
        }

        if (disposing)
        {
            // Dispose owned managed resources.
        }

        // Release directly owned unmanaged resources.

        _disposed = true;
    }
}
```

Microsoft's standard dispose pattern uses a public `Dispose()` plus a protected overridable `Dispose(bool)` for potentially inheritable classes.

Why Dispose(bool)?

The argument tells the cleanup code how it was reached.

EXAMPLE / FLOW

```
disposing == true
    →
Called from Dispose()
```

EXAMPLE / FLOW

```
disposing == false
    →
Called from finalizer
```

When disposing == true

You may clean up:

EXAMPLE / FLOW

```
Owned managed IDisposable resources

Direct unmanaged resources
```

When disposing == false

You should clean up:

EXAMPLE / FLOW

```
Direct unmanaged resources
```

but generally avoid depending on other managed objects, because finalization order isn't something you should rely on.

Example Inheritable Base Class

C#

```
public class GiftExportSession :
    IDisposable
{
    private StreamWriter?
```

```
        _writer;

    private bool _disposed;

    public GiftExportSession(
        string path)
    {
        _writer = new StreamWriter(path);
    }

    public void Dispose()
    {
        Dispose(
            true);

        GC.SuppressFinalize(
            this);
    }

    protected virtual void Dispose(
        bool disposing)
    {
        if (_disposed)
        {
            return;
        }

        if (disposing)
        {
            _writer?.Dispose();

            _writer = null;
        }

        _disposed = true;
    }
}
```

Why protected virtual?

If another class derives:

C#

```
public class CompressedGiftExportSession :
    GiftExportSession
```

the derived class may own additional resources.

It can override:

C#

```
protected override void Dispose(
    bool disposing)
```

and release its own resources.

Derived Dispose

Conceptually:

```C#
protected override void Dispose(
    bool disposing)
{
    if (disposing)
    {
        _ownedResource.Dispose();
    }

    base.Dispose(
        disposing);
}
```

The base part must also get a chance to release resources it owns.

Avoid Inheritable Disposable Types When Unnecessary

If a disposable type is not designed for inheritance:

```C#
public sealed class GiftExportSession :
    IDisposable
```

is often simpler.

Then you don't need a complicated overridable disposal hierarchy.

Finalizers

C# syntax:

```C#
~NativeResourceHolder()
{
}
```

This is a:

EXAMPLE / FLOW

```
Finalizer
```

Historically people sometimes call it a:

EXAMPLE / FLOW

```
Destructor
```

but `finalizer` is the more accurate .NET term.

Finalizers are automatically invoked by the runtime; they cannot be called directly, inherited, overloaded, or declared on structs.

Finalizers Are NOT Deterministic

You cannot predict:

EXAMPLE / FLOW

```
Exactly when finalizer runs
```

The GC determines that.

Therefore:

EXAMPLE / FLOW

```
Finalizer
≠
timely resource cleanup
```

Finalizers Are a Safety Net

If your type directly owns an unmanaged resource and the caller forgets:

C#

```
Dispose();
```

a finalizer can provide a last-resort cleanup mechanism.

But it should not be your main cleanup strategy.

Finalizer Example

Advanced example:

C#

```
public class NativeBuffer :
    IDisposable
{
    private bool _disposed;

    ~NativeBuffer()
    {
        Dispose(
            false);
    }

    public void Dispose()
    {
        Dispose(
            true);

        GC.SuppressFinalize(
            this);
    }

    protected virtual void Dispose(
        bool disposing)
    {
        if (_disposed)
        {
            return;
        }

        if (disposing)
        {
            // Dispose managed resources.
        }

        // Release native memory here.

        _disposed = true;
    }
}
```

GC.SuppressFinalize

When explicit disposal succeeds:

```
C#
Dispose();
```

there is no reason to later run the object's finalizer.

So the conventional pattern calls:

```
C#
GC.SuppressFinalize(
    this);
```

This removes the need for that object's finalizer to run later.

Why Finalizers Are Expensive

A finalizable object cannot usually be reclaimed as simply as an ordinary object.

It needs additional GC/finalization processing, which can delay memory reclamation.

Microsoft notes that finalization increases collection cost and can require an additional collection cycle before the object's memory is reclaimed.

Do Not Add Empty Finalizers

Bad:

```
C#
~GiftService()
{
}
```

This accomplishes nothing useful while making GC handling more expensive.

Do not create a finalizer unless your type directly owns a resource that truly requires one.

Most Application Classes Need No Finalizer

Typical types:

```
EXAMPLE / FLOW
Gift

GiftService

Campaign

Fund

GiftRepository wrapper over DI-managed DbContext
```

should not normally implement finalizers.

Prefer SafeHandle

If you directly work with native operating-system handles, .NET recommends wrapping them using:

rather than manually writing a finalizer whenever possible.

Why SafeHandle?

Native handle lifetime is surprisingly difficult to implement safely.

`SafeHandle` provides robust handle ownership/finalization behavior and integrates with platform invocation.

Conceptually:

EXAMPLE / FLOW

```
Raw native handle
    ↓
SafeHandle
    ↓
Your managed class
```

instead of:

EXAMPLE / FLOW

```
Raw IntPtr
    ↓
Your hand-written finalizer
```

Recommended Native Resource Model

EXAMPLE / FLOW

```
Native resource
    ↓
SafeHandle
    ↓
Your IDisposable class
```

Your class disposes:

EXAMPLE / FLOW

```
SafeHandle
```

and SafeHandle provides the difficult native-handle safety machinery.

Example Concept

C#

```
public sealed class NativeGiftFile :
    IDisposable
{
    private readonly SafeHandle
        _handle;

    public void Dispose()
    {
        _handle.Dispose();
    }
}
```

Actual SafeHandle subclasses depend on the native resource.

IAsyncDisposable

Some cleanup itself requires asynchronous I/O.

For this case .NET defines:

```
C#
IAsyncDisposable
```

Conceptually:

```
C#
public interface IAsyncDisposable
{
    ValueTask DisposeAsync();
}
```

`DisposeAsync` returns a `ValueTask` representing asynchronous cleanup.

Why Async Disposal?

Imagine cleanup requires:

```
EXAMPLE / FLOW
Flushing data over network

Finishing asynchronous stream operation

Committing/closing asynchronous infrastructure

Asynchronous protocol shutdown
```

A synchronous:

```
C#
Dispose()
```

cannot `await`.

So:

```
C#
DisposeAsync()
```

allows asynchronous completion.

await using

For `IAsyncDisposable`:

```
C#
await using (
    AsyncGiftWriter writer =
        await AsyncGiftWriter
            .CreateAsync())
{
    await writer.WriteAsync(
```

```
        gift);
}
```


When scope exits:

```
C#
await writer.DisposeAsync();
```

is performed.

The C# language specification describes `await using` as asynchronous `using` whose cleanup awaits `DisposeAsync`.

using vs await using

```
EXAMPLE / FLOW
IDisposable
    →
using
    →
Dispose()
```

```
EXAMPLE / FLOW
IAsyncDisposable
    →
await using
    →
await DisposeAsync()
```

Simple IAsyncDisposable Example

```
C#
public sealed class GiftBatchWriter :
    IAsyncDisposable
{
    private readonly StreamWriter
        _writer;

    private bool _disposed;

    public GiftBatchWriter(
        Stream stream)
    {
        _writer = new StreamWriter(stream);
    }

    public async ValueTask DisposeAsync()
    {
        if (_disposed)
        {
            return;
        }

        await _writer
            .DisposeAsync();

        _disposed = true;

        GC.SuppressFinalize(
            this);
```

```
    }
}
```


Use

```
C#
await using GiftBatchWriter writer = new(stream);

await writer.WriteAsync(
    gift);
```

At scope exit:

```
EXAMPLE / FLOW
DisposeAsync()
    ↓
await asynchronous cleanup
```

DisposeAsync Should Also Be Idempotent

Just like `Dispose`:

```
C#
await resource.DisposeAsync();

await resource.DisposeAsync();
```

should not normally perform dangerous duplicate cleanup.

Microsoft's `IAsyncDisposable` guidance states that subsequent disposal calls should be harmless and complete successfully.

Inheritable Async Dispose Pattern

For a non-sealed class, the standard pattern generally uses:

```
C#
protected virtual ValueTask
    DisposeAsyncCore()
```

with:

```
C#
public async ValueTask
    DisposeAsync()
```

Example:

```
C#
public class AsyncResourceHolder :
    IAsyncDisposable
{
    private IAsyncDisposable?
        _resource;

    public async ValueTask
        DisposeAsync()
    {
```

```
        await DisposeAsyncCore()
            .ConfigureAwait(
                false);

        GC.SuppressFinalize(
            this);
    }

    protected virtual async ValueTask
        DisposeAsyncCore()
    {
        if (_resource is not null)
        {
            await _resource
                .DisposeAsync()
                .ConfigureAwait(
                    false);

            _resource = null;
        }
    }
}
```

Microsoft recommends this `DisposeAsync` / `DisposeAsyncCore` shape when inheritance needs to be supported.

Class Implementing Both Interfaces

Some resources support:

C#

```
IDisposable
```

and:

C#

```
IAsyncDisposable
```

This gives callers a choice:

EXAMPLE / FLOW

```
Synchronous cleanup
or
Asynchronous cleanup
```

Example Concept

C#

```
public sealed class GiftTransferSession :
    IDisposable,
    IAsyncDisposable
{
    public void Dispose()
    {
        // Synchronous cleanup.
    }

    public async ValueTask
        DisposeAsync()
    {
```

```
        // Asynchronous cleanup.

        await Task.CompletedTask;
    }
}
```


Which Should Caller Use?

If asynchronous cleanup provides the intended nonblocking path:

```C#
await using GiftTransferSession session = new();
```

Otherwise:

```C#
using GiftTransferSession session = new();
```

The best choice depends on the type's API and your execution context.

Don't Call Both Manually

Avoid:

```C#
resource.Dispose();

await resource.DisposeAsync();
```

unless the type explicitly documents that workflow.

Choose the appropriate ownership/disposal mechanism.

Disposal Is About Ownership, Not Dependency Presence

Suppose:

```C#
public sealed class GiftService
{
    private readonly GiftDbContext
        _dbContext;

    public GiftService(
        GiftDbContext dbContext)
    {
        _dbContext = dbContext;
    }
}
```

Should:

```C#
GiftService
```

implement `IDisposable` just because:

EXAMPLE / FLOW

```
GiftDbContext is disposable?
```

TOPIC 19 OF 40

In ASP.NET Core DI:

EXAMPLE / FLOW

```
Usually no.
```

The service doesn't own the injected context's lifecycle.

The DI container does.

ASP.NET Core Dependency Injection Ownership

For services created by the built-in DI container, the container is responsible for invoking:

EXAMPLE / FLOW

```
Dispose()
```

or:

EXAMPLE / FLOW

```
DisposeAsync()
```

at the appropriate lifetime boundary.

Microsoft specifically says services resolved from the container should not be manually disposed by the consumer.

Scoped Disposal

Suppose:

C#

```
builder.Services
    .AddScoped<
        GiftDbContext>();
```

In a normal request:

EXAMPLE / FLOW

```
HTTP request begins
    ↓
DI scope created
    ↓
GiftDbContext created
    ↓
Request uses it
    ↓
Request scope ends
    ↓
Container disposes GiftDbContext
```

Singleton Disposal

For a DI-created singleton:

EXAMPLE / FLOW

```
Application starts
    ↓
Singleton created
    ↓
Used across application
    ↓
Host shuts down
    ↓
Container disposes singleton
```


Transient Disposal

Disposable transient services created by the container are also tracked for disposal according to the scope from which they are resolved.

This can become problematic if disposable transients are repeatedly resolved from the root container because they can remain rooted until the root provider is disposed. Microsoft calls this out as a DI anti-pattern.

Do Not Dispose Injected Dependencies

Bad:

C#

```
public sealed class GiftService :
    IDisposable
{
    private readonly GiftDbContext
        _dbContext;

    public GiftService(
        GiftDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public void Dispose()
    {
        _dbContext.Dispose();
    }
}
```

If DI owns:

EXAMPLE / FLOW

```
GiftDbContext
```

then `GiftService` should not independently dispose it.

That could prematurely invalidate the resource for other objects participating in the same scope.

Better

C#

```
public sealed class GiftService
{
    private readonly GiftDbContext
        _dbContext;
```

```
    public GiftService(
        GiftDbContext dbContext)
    {
        _dbContext = dbContext;
    }
}
```

Let DI manage disposal.

DI Ownership Rule

EXAMPLE / FLOW

```
Container created resource
    →
Container disposes resource
```

EXAMPLE / FLOW

```
You manually created resource
    →
You generally dispose resource
```

This is a powerful rule for ASP.NET Core applications.

Manually Creating a Scope

Sometimes background work needs its own scope:

C#

```
using IServiceScope scope = scopeFactory.CreateScope();

GiftService service = scope.ServiceProvider.GetRequiredService< GiftService>();

await service.ProcessAsync(...);
```

When:

EXAMPLE / FLOW

```
scope.Dispose()
```

runs:

EXAMPLE / FLOW

```
Scoped services created
inside that scope
are disposed
```

Async Scope

If async-disposable resources are involved:

C#

```
await using AsyncServiceScope scope = scopeFactory.CreateAsyncScope();
```

Then async disposal is available at scope end.

Do Not Keep Disposable Scoped Services Forever

Bad architecture:

```
EXAMPLE / FLOW
Singleton
    ↓
captures scoped DbContext
```

Now:

```
EXAMPLE / FLOW
Request-scoped resource
```

may effectively remain alive far longer than intended.

This is called:

```
EXAMPLE / FLOW
scope leakage / captive dependency
```

and can cause resource lifetime and concurrency problems.

Microsoft recommends scope validation to detect singleton-to-scoped dependency issues.

Resource Leak

A resource leak occurs when something remains allocated/held longer than intended.

Example:

```
C#
for (
    int i = 0;
    i < 10000;
    i++)
{
    FileStream stream = File.OpenRead(paths[i]);

    // Forgot Dispose.
}
```

Eventually you may exhaust:

```
EXAMPLE / FLOW
File handles

OS resources
```

even though managed memory itself can eventually be reclaimed.

Better

```
C#
foreach (
    string path
    in paths)
{
    using FileStream stream = File.OpenRead(path);
```

```
    Process(
        stream);
}
```


Each file handle is promptly released.

Exception-Safe Resource Management

This:

```C#
StreamWriter writer = new(path);

ProcessGift(
    writer);

writer.Dispose();
```

has a bug.

If:

```C#
ProcessGift(
    writer);
```

throws:

EXAMPLE / FLOW

```
writer.Dispose()
never runs
```

Correct

```C#
using StreamWriter writer = new(path);

ProcessGift(
    writer);
```

Even if `ProcessGift` throws:

EXAMPLE / FLOW

```
Dispose still happens
```

through the using/finally mechanism.

Dispose Should Not Be Used as a General "Reset"

Avoid designing:

```C#
resource.Dispose();

resource.StartAgain();
```

unless the type explicitly supports that lifecycle.

Ordinarily:

EXAMPLE / FLOW

```
Dispose
    →
End of usable lifetime
```

ObjectDisposedException Example

C#

```
public sealed class GiftWriter :
    IDisposable
{
    private bool _disposed;

    public void Write(
        Gift gift)
    {
        ObjectDisposedException
            .ThrowIf(
                _disposed,
                this);

        // Write.
    }

    public void Dispose()
    {
        _disposed = true;
    }
}
```

Finalizer vs Dispose

Dispose	Finalizer
Explicit	Runtime-controlled
Deterministic	Non-deterministic
Called by consumer/owner	Called by GC
Can dispose managed + unmanaged owned resources	Should focus on unmanaged cleanup
Immediate resource release	Last-resort safety mechanism
Preferred mechanism	Avoid unless necessary

SafeHandle vs Finalizer

Prefer:

EXAMPLE / FLOW

```
SafeHandle
```

over hand-written finalization for native handles.

Why?

EXAMPLE / FLOW

```
Reliable native handle release

Integrated finalization behavior

Fewer lifetime race bugs

Safer P/Invoke integration
```


Microsoft strongly recommends SafeHandle where a suitable wrapper can be used.

IDisposable vs IAsyncDisposable

Feature	IDisposable	IAsyncDisposable
Method	`Dispose()`	`DisposeAsync()`
Return	`void`	`ValueTask`
Usage	`using`	`await using`
Cleanup style	Synchronous	Asynchronous
Can await cleanup?	No	Yes
Typical use	Local/native/managed cleanup	Cleanup requiring async operations

Common Mistake — Relying on GC Instead of Dispose

Bad reasoning:

EXAMPLE / FLOW

```
It's managed .NET code,
so GC will handle everything.
```

GC handles managed memory.

It does not provide timely deterministic release for every external/native resource.

Common Mistake — Forgetting using

Bad:

C#

```
StreamReader reader = File.OpenText(path);

string text = reader.ReadToEnd();
```

If no explicit disposal occurs, resource lifetime may be unnecessarily extended.

Better:

C#

```
using StreamReader reader = File.OpenText(path);

string text = reader.ReadToEnd();
```

Common Mistake — Disposing Something You Don't Own

Bad:

```
C#
public void Process(
    Stream stream)
{
    // Work.

    stream.Dispose();
}
```

unless the method's contract explicitly transfers ownership.

Common Mistake — Forgetting Cascading Disposal

Class owns:

```
EXAMPLE / FLOW
StreamWriter
```

but:

```
C#
public void Dispose()
{
}
```

does nothing.

The outer type's disposal contract is incomplete.

Common Mistake — Double Dispose Assumptions

Consumers may call:

```
C#
Dispose()
```

more than once.

Your disposal implementation should normally tolerate repeated calls.

Common Mistake — Continuing to Use Disposed Object

```
C#
writer.Dispose();

writer.WriteLine(
    "Gift");
```

The resource's usable lifetime is finished.

Common Mistake — Writing Finalizers Everywhere

Bad:

```
C#
~GiftService()
{
}
```

Finalizers add GC overhead and should only exist for specific unmanaged-resource scenarios.

Common Mistake — Finalizer Disposes Managed Objects

Avoid depending on other managed objects from finalization logic.

Their finalization/lifetime state isn't something your finalizer should assume.

Use:

```
EXAMPLE / FLOW
Dispose(true)
    →
managed + unmanaged cleanup
```

```
EXAMPLE / FLOW
Dispose(false)
    →
unmanaged cleanup only
```

Common Mistake — Forgetting GC.SuppressFinalize

If a class has a finalizer and explicit disposal has already released the resource, suppress the unnecessary later finalizer.

```
C#
GC.SuppressFinalize(
    this);
```

Common Mistake — Raw IntPtr Instead of SafeHandle

If a suitable SafeHandle design exists, prefer it.

Hand-written native lifetime management is complex and security-sensitive.

Common Mistake — Using `using` on DI-Injected Service

Bad:

```
C#
public async Task ProcessAsync()
{
    using (
        _dbContext)
    {
    }
}
```

if `_dbContext` came from ASP.NET Core DI.

The container owns it and will dispose it according to its scope.

Common Mistake — Receiver Implements IDisposable Because Dependency Does

This:

EXAMPLE / FLOW

```
GiftService receives DbContext
```

does not mean:

EXAMPLE / FLOW

```
GiftService must implement IDisposable.
```

Ownership determines disposal responsibility.

Microsoft's DI guidance explicitly notes that merely receiving an `IDisposable` dependency does not require the receiver to become disposable.

Common Mistake — Wrong using Scope

C#

```
using StreamReader reader = File.OpenText(path);

// Hundreds of lines
// where reader is no longer needed.
```

The resource stays alive longer than necessary.

Keep disposal scopes narrow.

Common Mistake — Forgetting await using

If a resource requires asynchronous disposal:

C#

```
await using AsyncResource resource = await CreateAsync();
```

is usually preferable to accidentally using only a synchronous cleanup path.

Common Mistake — Fire-and-Forget DisposeAsync

Bad:

C#

```
resource.DisposeAsync();
```

without awaiting/observing completion.

Better:

C#

```
await resource.DisposeAsync();
```

or:

C#

```
await using ...
```

Common Mistake — Calling GC.Collect to Force Cleanup

Do not write:

```
C#
GC.Collect();
```

just because you forgot to dispose resources.

Garbage collection is not a replacement for deterministic disposal.

Finalization itself is non-deterministic, and forcing GC can harm performance.

Common Mistake — Assuming Dispose Frees Object Memory

Dispose releases resources.

Memory occupied by the managed object itself is still reclaimed by the GC later.

```
EXAMPLE / FLOW
Dispose
    ≠
free managed object immediately
```

Common Mistake — Dispose and Finalizer Do Same Work Blindly

The two paths have different safety assumptions.

```
EXAMPLE / FLOW
Dispose(true)
    →
managed cleanup safe
```

```
EXAMPLE / FLOW
Dispose(false)
    →
managed object graph may not be safely usable
```

Design accordingly.

Common Mistake — Catching and Ignoring Dispose Failures

Cleanup failures can matter.

Do not automatically write:

```
C#
try
{
    resource.Dispose();
}
catch
{
}
```

unless suppressing that failure is explicitly justified.

You may hide:

EXAMPLE / FLOW

```
Incomplete flush

Data corruption

Resource-release failure
```


Performance Considerations

Deterministic disposal often improves resource efficiency.

For example:

EXAMPLE / FLOW

```
Open 1 file
    ↓
Use
    ↓
Dispose immediately
```

allows the handle to be reused quickly.

Without disposal:

EXAMPLE / FLOW

```
Many resources stay open
    ↓
Handle pressure
    ↓
System resource exhaustion
```

Finalizer Cost

Finalizable objects require more GC processing and tend to remain alive longer before memory reclamation.

Therefore:

EXAMPLE / FLOW

```
Avoid unnecessary finalizers.
```

Microsoft explicitly warns that finalization imposes significant lifetime/GC overhead.

Async Disposal Performance

Async disposal is useful when cleanup itself waits for asynchronous operations.

But don't create:

C#

```
IAsyncDisposable
```

just because:

EXAMPLE / FLOW

```
Async sounds faster.
```

If cleanup is entirely synchronous:

is usually sufficient.

Security Considerations

Resource lifetime can become a security issue.

Example:

EXAMPLE / FLOW

```
Open sensitive file

Read secret

Forget to release handle
```

may leave resources active longer than intended.

Similarly native handles can have subtle reuse/race vulnerabilities; SafeHandle exists partly to make native handle lifetime safer.

Secrets and Buffers

Disposal does not automatically mean:

EXAMPLE / FLOW

```
All sensitive bytes
were cryptographically erased
from every memory location.
```

Do not assume:

C#

```
Dispose()
```

is a universal secret-erasure mechanism.

Sensitive-data lifecycle requires API-specific handling.

Database Transactions

A transaction should be scoped carefully:

C#

```
await using var transaction =
    await dbContext.Database
        .BeginTransactionAsync(
            cancellationToken);
```

Then:

EXAMPLE / FLOW

```
Use transaction

Commit if successful

Dispose when scope ends
```

Resource ownership should be explicit.

Resource Lifetime and Locks

Holding a lock/resource longer than necessary can cause:

EXAMPLE / FLOW

```
Contention

Deadlocks

Reduced throughput
```

This is another reason to keep resource scopes narrow.

Fundraising Example — Export File

C#

```
public sealed class GiftCsvExporter
{
    public void Export(
        string path,
        IEnumerable<Gift> gifts)
    {
        using StreamWriter writer = new(path);

        writer.WriteLine(
            "GiftId,Amount");

        foreach (
            Gift gift
            in gifts)
        {
            writer.WriteLine(
                $"{gift.GiftId},{gift.Amount}");
        }
    }
}
```

Ownership:

EXAMPLE / FLOW

```
GiftCsvExporter.Export
    →
creates StreamWriter
    →
owns StreamWriter
    →
using disposes StreamWriter
```

Fundraising Example — Borrowed Writer

C#

```
public static void WriteGift(
    TextWriter writer,
    Gift gift)
{
    ArgumentNullException.ThrowIfNull(
        writer);
```

```
    ArgumentNullException.ThrowIfNull(
        gift);

    writer.WriteLine(
        $"{gift.GiftId},{gift.Amount}");
}
```

This method:

EXAMPLE / FLOW

```
Uses writer
```

but does not:

EXAMPLE / FLOW

```
Own writer
```

Therefore it should not dispose it.

Fundraising Example — Owned Disposable Resource

C#

```
public sealed class GiftExportSession :
    IDisposable
{
    private readonly StreamWriter
        _writer;

    private bool _disposed;

    public GiftExportSession(
        string path)
    {
        _writer = new StreamWriter(path);
    }

    public void Write(
        Gift gift)
    {
        ObjectDisposedException
            .ThrowIf(
                _disposed,
                this);

        _writer.WriteLine(
            $"{gift.GiftId},{gift.Amount}");
    }

    public void Dispose()
    {
        if (_disposed)
        {
            return;
        }

        _writer.Dispose();

        _disposed = true;

        GC.SuppressFinalize(
            this);
```

```
    }
}
```


Why No Finalizer Here?

Because:

EXAMPLE / FLOW

`GiftExportSession`

does not directly own a raw unmanaged resource.

It owns:

EXAMPLE / FLOW

`StreamWriter`

which already implements the appropriate cleanup mechanisms.

Adding your own finalizer would be unnecessary.

Use

C#

```
using GiftExportSession session = new("gifts.csv");

session.Write(
    gift1);

session.Write(
    gift2);
```

Fundraising Example — Async Export Session

C#

```
public sealed class AsyncGiftExportSession :
    IAsyncDisposable
{
    private readonly StreamWriter
        _writer;

    private bool _disposed;

    public AsyncGiftExportSession(
        Stream stream)
    {
        _writer = new StreamWriter(stream);
    }

    public async Task WriteAsync(
        Gift gift)
    {
        ObjectDisposedException
            .ThrowIf(
                _disposed,
                this);

        await _writer.WriteLineAsync(
            $"{gift.GiftId},{gift.Amount}");
    }
```

```
    public async ValueTask
        DisposeAsync()
    {
        if (_disposed)
        {
            return;
        }

        await _writer
            .DisposeAsync();

        _disposed = true;

        GC.SuppressFinalize(
            this);
    }
}
```

Usage:

C#
```
await using AsyncGiftExportSession session = new(stream);

await session.WriteAsync(
    gift);
```

Practical Exercise 1 — Basic using

Rewrite:

C#
```
StreamReader reader = File.OpenText(path);

string content = reader.ReadToEnd();

reader.Dispose();
```

as:

C#
```
using StreamReader reader = File.OpenText(path);

string content = reader.ReadToEnd();
```

Now cleanup remains exception-safe.

Practical Exercise 2 — Ownership

Method receives:

C#
```
public void Import(
    Stream stream)
```

Who should dispose `stream`?

Default answer:

EXAMPLE / FLOW

```
The caller/owner,
unless the method contract explicitly
takes ownership.
```


Practical Exercise 3 — Cascading Disposal

Given:

C#

```
public sealed class A :
    IDisposable
{
    private readonly B
        _b = new();
}
```

and:

C#

```
B : IDisposable
```

implement:

C#

```
public void Dispose()
{
    _b.Dispose();
}
```

because A owns B.

Practical Exercise 4 — Idempotence

Bad:

C#

```
public void Dispose()
{
    _handle.Close();

    throw new InvalidOperationException(
        "Already closed");
}
```

Better behavior:

EXAMPLE / FLOW

```
First Dispose
    →
cleanup

Later Dispose
    →
return harmlessly
```

Practical Exercise 5 — DI Ownership

```C#
public GiftService(
    GiftDbContext dbContext)
```

Should GiftService dispose it?

EXAMPLE / FLOW

```
No,
if ASP.NET Core DI created and injected it.
```

The container owns that lifetime.

Practical Exercise 6 — await using

Given:

```C#
IAsyncDisposable resource = CreateResource();
```

Use:

```C#
await using (
    resource)
{
    // Async resource usage.
}
```

so `DisposeAsync()` is awaited at scope exit.

Practical Exercise 7 — Finalizer Decision

Class only owns:

```C#
StreamWriter
```

Should it have a finalizer?

EXAMPLE / FLOW

```
No.
```

Dispose the StreamWriter.

Practical Exercise 8 — Native Handle

Class directly owns:

EXAMPLE / FLOW

```
OS native handle
```

Preferred approach:

rather than immediately building a custom finalizer.

Practical Exercise 9 — Object After Dispose

```C#
resource.Dispose();

resource.Write();
```

Correct design may throw:

EXAMPLE / FLOW
ObjectDisposedException

because operational lifetime ended.

Practical Exercise 10 — Scope

Which is better?

```C#
using StreamReader reader = File.OpenText(path);

// 300 unrelated lines.
```

or:

```C#
using (
    StreamReader reader =
        File.OpenText(
            path))
{
    // Only required operations.
}
```

If reader is needed only in that block:

EXAMPLE / FLOW
Second version

gives a narrower resource lifetime.

Interview Questions

1. What is IDisposable?

`IDisposable` defines the `Dispose()` cleanup contract for resources requiring deterministic release.

2. Why do we need IDisposable if .NET has garbage collection?

Because GC manages managed memory but doesn't provide deterministic cleanup of external/unmanaged resources such as native handles.

3. What is deterministic cleanup?

Releasing a resource at a known point rather than waiting for an unpredictable GC/finalization cycle.

4. What does `Dispose()` do?

It releases or resets resources owned by the object.

It does not directly free the managed object's memory.

5. What does `using` do?

It ensures an eligible disposable resource is disposed when control leaves the scope, including exceptional exits.

6. How is using implemented conceptually?

Through:

EXAMPLE / FLOW

```
resource acquisition

try

finally

Dispose
```

semantics.

7. What is a using declaration?

Syntax such as:

C#

```
using StreamReader reader = File.OpenText(path);
```

The resource is disposed when the containing scope ends.

8. What is the difference between a using statement and declaration?

Statement:

EXAMPLE / FLOW

```
Resource lifetime
ends at block end.
```

Declaration:

EXAMPLE / FLOW

```
Resource lifetime
ends at enclosing scope end.
```

9. In what order are multiple using resources disposed?

Reverse order of declaration.

10. Can a using variable be reassigned?

No.

Variables declared by a using statement/declaration are readonly for the relevant language rules.

11. Who should dispose an IDisposable object?

Usually whoever owns its lifetime.

Often:

EXAMPLE / FLOW

```
Creator
    →
Owner
    →
Disposer
```

but framework ownership can override that rule.

12. Should a method dispose an IDisposable parameter it receives?

Usually no unless the method's API explicitly takes ownership.

13. What is cascading disposal?

An owner disposes disposable resources it owns.

If A owns B and B owns C:

EXAMPLE / FLOW

```
A.Dispose()
    →
B.Dispose()
    →
C.Dispose()
```

Microsoft recommends propagating cleanup through such ownership chains.

14. Should Dispose be safe to call more than once?

Yes, disposal implementations should normally be idempotent.

15. What exception can be thrown when using an already disposed object?

C#

```
ObjectDisposedException
```

16. What is the classic dispose pattern?

For an inheritable disposable type:

EXAMPLE / FLOW

```
Dispose()
    ↓
Dispose(bool)
    ↓
GC.SuppressFinalize(this)
```

with an optional finalizer for directly owned unmanaged resources.

17. What does the `disposing` bool mean?

EXAMPLE / FLOW

```
true
    →
Called through Dispose;
managed resource cleanup is allowed.

false
    →
Called through finalizer;
focus on direct unmanaged cleanup.
```

18. What is a finalizer?

A special class member invoked automatically by the runtime during finalization for eligible objects.

It provides last-resort unmanaged cleanup.

19. Can structs have finalizers?

No.

Finalizers are only available on classes.

20. Can you call a finalizer manually?

No.

The runtime invokes it automatically.

21. When should a class have a finalizer?

Only in advanced scenarios where the type directly owns unmanaged resources that aren't already protected by a safe finalizable wrapper.

Prefer `SafeHandle` where possible.

22. Why are finalizers expensive?

Finalizable objects require additional GC/finalization processing and generally survive longer before memory can be reclaimed.

23. What does GC.SuppressFinalize do?

It tells the runtime that finalization is no longer required for that object because deterministic cleanup has already happened.

24. What is SafeHandle?

`SafeHandle` is a managed abstraction for safely owning/releasing native handles and provides robust finalization behavior.

Microsoft recommends it over hand-writing native-handle finalizers when possible.

25. What is IAsyncDisposable?

An asynchronous resource-cleanup contract:

C#

```
ValueTask DisposeAsync();
```

26. What is `await using`?

The asynchronous form of `using`.

It awaits:

```
C#
DisposeAsync()
```

when scope exits.

27. Why would disposal need to be asynchronous?

Cleanup may itself require asynchronous I/O, such as flushing or shutting down an asynchronous resource.

28. What is DisposeAsyncCore?

For inheritable `IAsyncDisposable` classes, it is the conventional protected virtual method containing extensible asynchronous cleanup logic.

29. Can a class implement both IDisposable and IAsyncDisposable?

Yes.

It can offer both synchronous and asynchronous cleanup paths.

30. Should you call Dispose and DisposeAsync both?

Usually no.

Use the cleanup mechanism appropriate for the type and context unless the API explicitly documents otherwise.

31. Should an ASP.NET Core service dispose an injected DbContext?

No, when the DI container created and injected it.

The container owns that service's disposal.

32. Who disposes scoped DI services?

The container disposes them when the scope ends, typically at request end in a web application.

33. Who disposes singleton DI services?

The container disposes DI-created singleton instances when the service container is disposed, normally during application shutdown.

34. Does receiving IDisposable through DI mean my class must implement IDisposable?

No.

Microsoft specifically states that receiving a disposable dependency doesn't require the receiver to implement `IDisposable`, and the receiver shouldn't dispose a DI-owned dependency.

35. What is a resource leak?

A resource remains acquired longer than intended because cleanup didn't happen correctly.

Examples:

EXAMPLE / FLOW

```
Open file handles

Sockets

Connections

Native memory
```


36. Does Dispose immediately remove the object from memory?

No.

The managed object remains subject to normal GC lifetime rules.

Dispose ends its resource ownership/useful lifetime.

37. Why shouldn't every class implement IDisposable?

Because most classes own no resources requiring deterministic cleanup.

Adding disposal without need increases lifecycle complexity.

38. Is Dispose only for unmanaged resources?

Its primary motivation is deterministic resource cleanup, especially unmanaged resources, but a type also commonly implements it when it owns disposable managed resources or other deterministic registrations/locks that must be released.

39. Is finalization guaranteed at application shutdown?

No.

Modern .NET doesn't guarantee that every finalizer runs as part of process termination.

Explicit disposal should be used when cleanup must occur predictably.

40. Should you call GC.Collect to dispose resources sooner?

No.

Correct ownership plus `Dispose`, `using`, or `await using` should manage resource lifetime.

Forcing collection is not a replacement for deterministic cleanup.

Best Practices

Use:

C#

```
using
```

for synchronously disposable resources you own.

Use:

C#

```
await using
```

for asynchronously disposable resources you own.

Keep resource lifetimes as short as practical.

Think explicitly about ownership:

EXAMPLE / FLOW

```
Who created it?

Who owns it?

Who disposes it?
```

Do not dispose borrowed objects unless ownership was transferred.

Cascade disposal through owned disposable objects.

Make `Dispose` and `DisposeAsync` idempotent.

After disposal, guard operational methods with:

C#

```
ObjectDisposedException
```

when appropriate.

Use a simple disposal implementation for sealed classes when inheritance isn't needed.

Use the complete dispose pattern when designing an inheritable disposable base class.

Only implement finalizers for advanced direct unmanaged-resource scenarios.

Prefer:

C#

```
SafeHandle
```

over writing native-resource finalizers yourself.

Call:

C#

```
GC.SuppressFinalize(
    this);
```

when explicit disposal makes a finalizer unnecessary.

Do not assume:

EXAMPLE / FLOW

```
Dispose
=
GC
```

They solve different problems.

Do not assume:

EXAMPLE / FLOW

```
Dispose
=
object memory instantly freed.
```

Avoid unnecessary finalizers because they increase GC overhead.

In ASP.NET Core:

EXAMPLE / FLOW

```
DI-created resource
    →
DI container owns disposal.
```


Do not manually dispose injected scoped/singleton services.

Avoid singleton-to-scoped lifetime capture.

Avoid resolving disposable transient services repeatedly from the root provider.

Do not implement `IDisposable` merely because one injected dependency is disposable.

Use asynchronous disposal only when cleanup genuinely benefits from asynchronous operations.

Do not ignore disposal failures casually.

Quick Revision

Concept	Meaning
Garbage Collection	Reclaims managed memory
Disposal	Releases resources deterministically
`IDisposable`	Synchronous cleanup contract
`Dispose()`	Releases owned resources
`using`	Ensures synchronous disposal
using declaration	Disposes at scope end
`IAsyncDisposable`	Async cleanup contract
`DisposeAsync()`	Performs async cleanup
`await using`	Awaits async disposal
Ownership	Determines who must dispose
Cascading disposal	Owner disposes owned children
Idempotent	Safe to dispose repeatedly
`ObjectDisposedException`	Operation attempted after disposal
Finalizer	GC-driven last-resort cleanup
`GC.SuppressFinalize`	Prevent unnecessary finalizer
`SafeHandle`	Preferred native-handle wrapper
`Dispose(bool)`	Classic extensible dispose pattern
`DisposeAsyncCore()`	Extensible async dispose pattern
DI ownership	Container disposes services it creates

Complete Fundraising Example

Consider a batch receipt export.

We want:

EXAMPLE / FLOW

```
Open output resource
    ↓
Write many Gift receipts
    ↓
Flush
    ↓
Release resource
```


Receipt Export Writer

C#

```
public sealed class GiftReceiptExport :
    IAsyncDisposable
{
    private readonly StreamWriter
        _writer;

    private bool _disposed;

    public GiftReceiptExport(
        Stream output)
    {
        ArgumentNullException.ThrowIfNull(
            output);

        _writer = new StreamWriter(output);
    }

    public async Task WriteAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        ObjectDisposedException
            .ThrowIf(
                _disposed,
                this);

        ArgumentNullException.ThrowIfNull(
            gift);

        string line =
            $"{gift.GiftId},"
            +
            $"{gift.ConstituentId},"
            +
            $"{gift.TotalAmount:F2}";

        await _writer.WriteLineAsync(
            line.AsMemory(),
            cancellationToken);
    }

    public async ValueTask
        DisposeAsync()
    {
        if (_disposed)
        {
            return;
        }

        await _writer
```

```
            .DisposeAsync();

        _disposed = true;

        GC.SuppressFinalize(
            this);
    }
}
```

Consumer

C#

```
public async Task ExportAsync(
    Stream output,
    IEnumerable<Gift> gifts,
    CancellationToken cancellationToken)
{
    await using GiftReceiptExport exporter = new(output);

    foreach (
        Gift gift
        in gifts)
    {
        await exporter.WriteAsync(
            gift,
            cancellationToken);
    }
}
```

Flow:

EXAMPLE / FLOW

```
ExportAsync
    ↓
Create GiftReceiptExport
    ↓
Create StreamWriter
    ↓
Write Gift 1
    ↓
Write Gift 2
    ↓
Write Gift N
    ↓
Scope exits
    ↓
await DisposeAsync()
    ↓
StreamWriter disposed
```

Ownership in This Example

`GiftReceiptExport` creates:

EXAMPLE / FLOW

```
StreamWriter
```

therefore:

and disposes it.

But it receives:

EXAMPLE / FLOW
Stream output

from the caller.

Whether disposing the writer should also close that underlying stream is an API ownership decision.

A production implementation may expose something like:

EXAMPLE / FLOW
leaveOpen

when callers must retain ownership of the supplied stream.

DI Example

C#
```
public sealed class GiftService
{
    private readonly FundraisingDbContext
        _dbContext;

    public GiftService(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public Task<Gift?>
        FindAsync(
            long giftId,
            CancellationToken cancellationToken)
    {
        return _dbContext.Gifts
            .FindAsync(
                [giftId],
                cancellationToken)
            .AsTask();
    }
}
```

Do **not** automatically add:

C#
```
IDisposable
```

to `GiftService`.

If:

EXAMPLE / FLOW
FundraisingDbContext

came from ASP.NET Core DI:

EXAMPLE / FLOW

```
DI container owns its lifetime.
```


The service merely borrows it for the scope.

Complete Ownership Flow

EXAMPLE / FLOW

```
Who created/acquired resource?
        ↓
Your code?
        ├─ Yes
        │    ↓
        │ Usually your code owns disposal
        │
        └─ No
             ↓
        Was it supplied by framework/DI/caller?
             ↓
        Determine ownership contract
```

Disposal Choice Flow

EXAMPLE / FLOW

```
Resource needs deterministic cleanup?
        ↓
No
    →
No IDisposable needed
```

EXAMPLE / FLOW

```
Yes
    ↓
Cleanup synchronous?
        ↓
Yes
    →
IDisposable
    →
using
```

EXAMPLE / FLOW

```
Cleanup requires async work?
        ↓
Yes
    →
IAsyncDisposable
    →
await using
```

Implementation Flow

EXAMPLE / FLOW

```
Sealed type
    ↓
Owns managed disposable resources
```

```
    ↓
Simple Dispose / DisposeAsync
```

EXAMPLE / FLOW

```
Inheritable disposable type
    ↓
Dispose()
    ↓
protected virtual Dispose(bool)
```

EXAMPLE / FLOW

```
Inheritable async-disposable type
    ↓
DisposeAsync()
    ↓
protected virtual DisposeAsyncCore()
```

Native Resource Flow

EXAMPLE / FLOW

```
Direct unmanaged/native handle
        ↓
Can SafeHandle represent it?
        ↓
Yes
    →
Use SafeHandle
    →
Dispose SafeHandle
```

Only in advanced cases:

EXAMPLE / FLOW

```
Need direct custom finalization
    ↓
Finalizer
    ↓
Dispose(false)
```

ASP.NET Core Flow

EXAMPLE / FLOW

```
HTTP Request
    ↓
DI creates scope
    ↓
DbContext / scoped services created
    ↓
Controller + GiftService use them
    ↓
Request completes
    ↓
Scope ends
    ↓
DI container calls
Dispose / DisposeAsync
```

Consumer classes should not fight that lifecycle by manually disposing the injected dependencies.

Final Mental Model

Remember:

EXAMPLE / FLOW

```
GC
    →
Managed memory
```

EXAMPLE / FLOW

```
Dispose
    →
Resource lifetime
```

EXAMPLE / FLOW

```
using
    →
Automatic synchronous Dispose
```

EXAMPLE / FLOW

```
await using
    →
Automatic awaited DisposeAsync
```

EXAMPLE / FLOW

```
Finalizer
    →
Last-resort unmanaged cleanup
```

EXAMPLE / FLOW

```
SafeHandle
    →
Preferred native handle safety
```

EXAMPLE / FLOW

```
DI-created dependency
    →
DI owns disposal
```

The most important lesson is:

Resource management in C# is primarily an ownership problem. The garbage collector reclaims managed memory, but resources with deterministic lifetime requirements should be disposed explicitly. Use `using` or `await using`, dispose only resources you own, cascade cleanup through owned objects, keep disposal idempotent, prefer `SafeHandle` over hand-written finalizers for native handles, and let ASP.NET Core's DI container dispose the services it creates.

For the Fundraising Platform, the practical default is:

EXAMPLE / FLOW

```
Gift / Campaign / Fund entities
    →
No IDisposable merely because
```

```
they are managed objects

File/stream export created locally
    →
using / await using

Custom wrapper owning disposable resource
    →
IDisposable / IAsyncDisposable

DI-injected DbContext
    →
Do not manually dispose

Direct native handle
    →
SafeHandle where possible
```

Next roadmap topic: Topic 20 — Generics.